

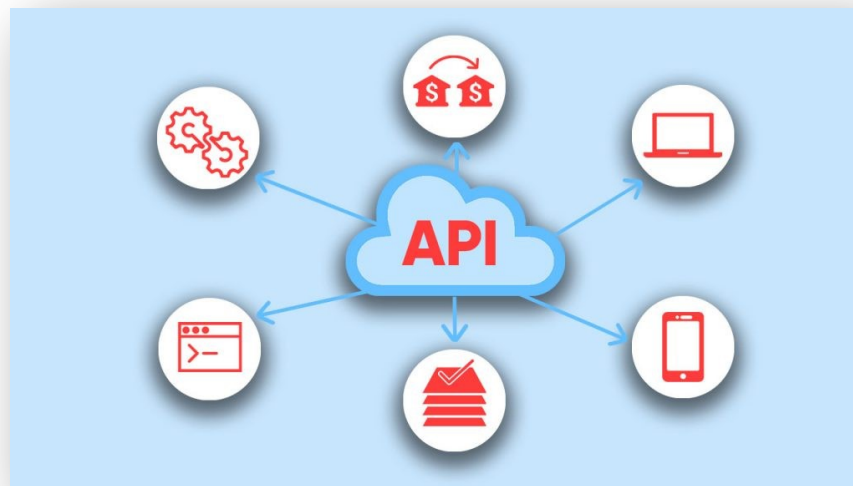
REST API

Desenvolvimento de Software para Web 1

Material baseado nos slides gentilmente disponibilizados pelo prof. Alan Valejo

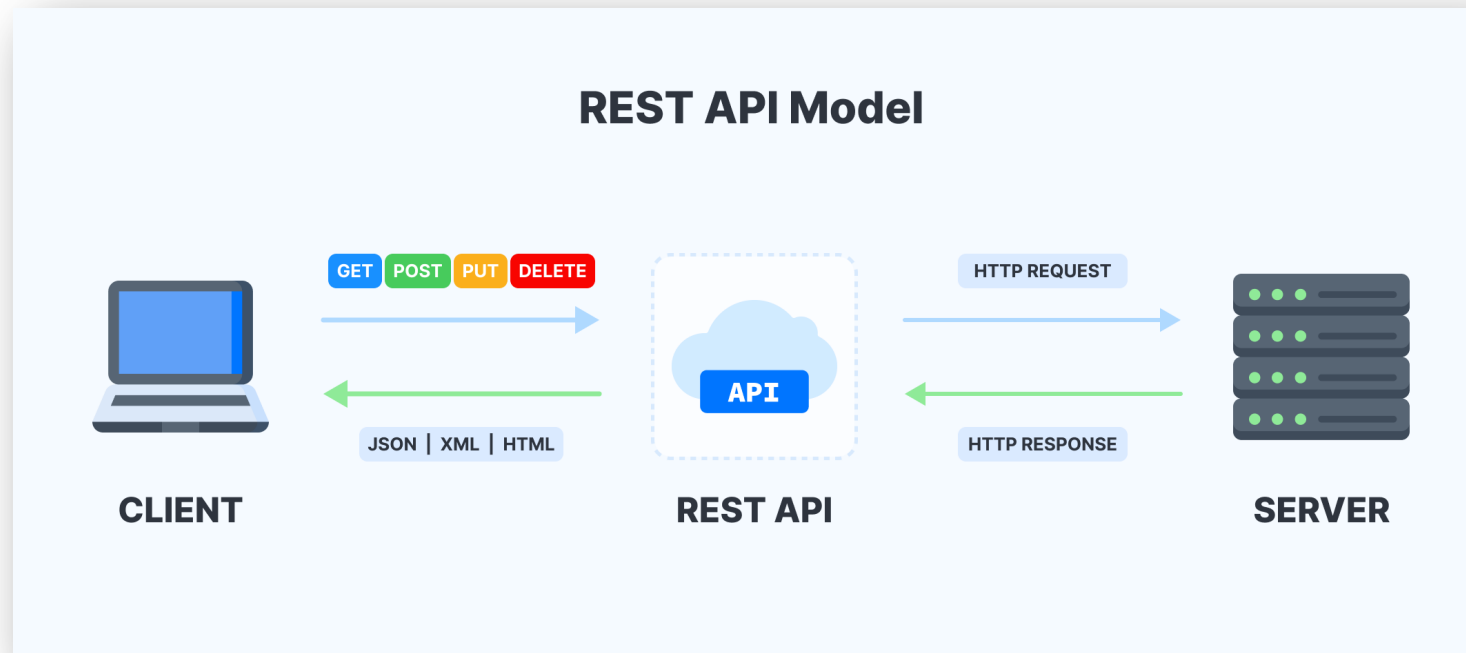
- Motivação
- REST API
- Spring MVC REST API

- Porque APIs (*Application Programming Interface*, ou Interface de Programação de Aplicação) são tão importantes?
 - Uma API é um **contrato** fornecido por um software para outro.
 - É uma maneira de dois sistemas de software trocarem informações.



O que é REST API?

- Essa arquitetura foi criada pelo cientista da computação Roy Fielding em sua dissertação de doutorado, chamada de “*Architectural Styles and the Design of Network-based Software Architectures*”, publicada em 2000 pela Universidade da Califórnia.
- API REST é uma abstração de arquitetura de software que fornece dados em um formato padronizado para modelos de requisições HTTP.



- REST é um acrônimo para:
 - **RE**presentational
 - **S**tate
 - **T**ransfer
- REST é um estilo de arquitetura baseado em padrões Web e no protocolo HTTP.
- Permite a construção de aplicações Web seguindo boas práticas.
- Em termos gerais:
 1. Em uma arquitetura baseada em REST, **tudo é um recurso**.
 2. Um **recurso** é acessado por meio de uma interface comum baseada nos métodos padrão HTTP.
- Portanto, você tem um **servidor REST** que fornece **acesso aos recursos** e um **cliente REST** que acessa e modifica os recursos REST.

- Três pontos importantes:
 1. Cada recurso deve suportar as operações comuns do protocolo HTTP
 - GET, POST, PUT, DELET ...
 2. Os recursos são identificados por IDs globais
 - Normalmente são URIs ou URLs
 - `http://localhost:8080/Aplicacao/users`
 3. REST permite que recursos tenham diferentes representações
 - HTML, XML, JSON etc.

- Os métodos POST, GET, PUT e DELETE são normalmente usados em arquiteturas REST para operações do CRUD

Método HTTP	Operação CRUD	Descrição
POST	CREATE	Adiciona um novo recurso
GET	READ	Acessa 1 ou vários recurso(s)
PUT	UPDATE	Atualiza um recurso
DELETE	DELETE	Remove um recurso

Método HTTP	URI	Descrição
POST	/cidades	Adiciona uma nova cidade
GET	/cidades	Retorna todas as cidades
GET	/cidades/{id}	Retorna a cidade de id = {id}
GET	/cidades/estados/{id}	Retorna as cidades do estado de id = {id}
PUT	/cidades/{id}	Atualiza a cidade de id = {id}
DELETE	/cidades/{id}	Remove a cidade de id = {id}

- Processo pelo qual o cliente determina a representação do recurso
 1. Através do HTTP header
 - Accept: application/json
 2. Através da extensão na URL (opcionalmente)
 - Não recomendado atualmente pela comunidade de desenvolvedores
 - `http://localhost:8080/restService/persons.json`



- Acessar:
- https://pt.wikipedia.org/wiki/Lista_de_c%C3%B3digos_de_estado_HTTP

- Não misture plural e singular. Mantenha simples e use apenas o **plural** para todos os recursos.

Purpose	Method	Incorrect	Correct
Retrieves a list of users	GET	/getAllUsers	/users
Create a new user	POST	/createUser	/users
Delete a user	DELETE	/deleteUser	/users/10
Get balance of user	GET	/getUserBalance	/users/11/balance

O *delete* está implícito no método

- Spring MVC é adequado para criar serviços web (REST API) porque é baseado no mapeamento de URLs a solicitações (controladores) e por poder atender com flexibilidade a diferentes tipos de conteúdo
- Recursos são modelos (entidades – classes DTO) associados a controladores REST (@RestController)
- Integra-se bem com uma série de serializadores, tanto para JSON (Jackson) quanto para XML (JAXB)

Método HTTP	URI	Descrição
POST	/cidades	Adiciona uma nova cidade
GET	/cidades	Retorna todas as cidades
GET	/cidades/{id}	Retorna a cidade de id = {id}
GET	/cidades/estados/{id}	Retorna as cidades do estado de id = {id}
PUT	/cidades/{id}	Atualiza a cidade de id = {id}
DELETE	/cidades/{id}	Remove a cidade de id = {id}

Método HTTP	URI	Descrição
POST	/cidades	Adiciona uma nova cidade

```
@CrossOrigin
@RestController
public class CidadeRestController {

    private ICidadeService service; 10 usages

    public CidadeRestController(ICidadeService service) {
        this.service = service;
    }

    @PostMapping(path = "/cidades")
    public ResponseEntity<Cidade> cria(@Valid @RequestBody Cidade cidade, BindingResult result) {
        if (result.hasErrors()) {
            return ResponseEntity.badRequest().body(null);
        } else {
            service.save(cidade);
            return ResponseEntity.ok(cidade);
        }
    }

    @GetMapping(path = "/cidades")
    public ResponseEntity<List<Cidade>> lista() {...}
}
```

Método HTTP	URI	Descrição
GET	/cidades	Retorna todas as cidades

```
@CrossOrigin
@RestController
public class CidadeRestController {

    private ICidadeService service; 10 usages

    public CidadeRestController(ICidadeService service) {
        this.service = service;
    }

    @PostMapping(path = "/cidades")
    public ResponseEntity<Cidade> cria(@Valid @RequestBody Cidade cidade, BindingResult result) {...}

    @GetMapping(path = "/cidades")
    public ResponseEntity<List<Cidade>> lista() {
        List<Cidade> lista = service.findAll();
        if (lista.isEmpty()) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(lista);
    }
}
```

Método HTTP	URI	Descrição
GET	/cidades/{id}	Retorna a cidade de id = {id}

```
@GetMapping(path = "{id}")
public ResponseEntity<Cidade> lista(@PathVariable("id") long id) {
    Cidade cidade = service.findById(id);
    if (cidade == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(cidade);
}
```

Método HTTP	URI	Descrição
GET	/cidades/estados/{id}	Retorna as cidades do estado de id = {id}

```
@GetMapping(path = "/cidades/estados/{id}")
public ResponseEntity<List<Cidade>> listaPorEstado(@PathVariable("id") long id) {
    List<Cidade> lista = service.findByEstado(id);

    if (lista.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(lista);
}
```


Método HTTP	URI	Descrição
PUT	/cidades/{id}	Atualiza a cidade de id = {id}

```
@PutMapping(path = "/cidades/{id}")
public ResponseEntity<Cidade> atualiza(@PathVariable("id") long id,
                                       @Valid @RequestBody Cidade cidade,
                                       BindingResult result) {

    if (result.hasErrors()) {
        return ResponseEntity.badRequest().body(null);
    } else {
        Cidade c = service.findById(id);
        if (c == null) {
            return ResponseEntity.notFound().build();
        } else {
            cidade.setId(id);
            service.save(cidade);
            return ResponseEntity.ok(cidade);
        }
    }
}
```

Método HTTP	URI	Descrição
DELETE	/cidades/{id}	Remove a cidade de id = {id}

```
@DeleteMapping(path = "{id}")
public ResponseEntity<Boolean> remove(@PathVariable("id") long id) {

    Cidade cidade = service.findById(id);
    if (cidade == null) {
        return ResponseEntity.notFound().build();
    } else {
        service.delete(id);
        return ResponseEntity.ok(body: true);
    }
}
```

- Spring MVC é adequado para criar serviços web (REST API) porque é baseado no mapeamento de URLs a solicitações (controladores) e por poder atender com flexibilidade a diferentes tipos de conteúdo
- Recursos são modelos (entidades) associados a controladores REST (@RestController)
- Integra-se bem com uma série de serializadores, tanto para JSON (Jackson) quanto para XML (JAXB)

```
{
  "id": 4601,
  "nome": "São Carlos",
  "estado": { "id": 26, "sigla": "SP", "nome": "São Paulo" }
}
```

```
public class Cidade {
    private Long id;
    private String nome;
    private Estado estado;

    public Cidade() {}
    public Cidade(String nome, Estado estado) {}
    public Long getId() {}
    public void setId(Long id) {}
    public String getNome() {}
    public void setNome(String nome) {}
    public Estado getEstado() {}
    public void setEstado(Estado estado) {}

    @Override
    public String toString() {
        return nome + "/" + estado.getSigla();
    }
}
```



```
public class Estado {
    private Long id;
    private String sigla;
    private String nome;

    public Estado() {}
    public Estado(Long id, String sigla, String nome) {}
    public Long getId() {}
    public void setId(Long id) {}
    public String getSigla() {}
    public void setSigla(String sigla) {}
    public String getNome() {}
    public void setNome(String nome) {}

    @Override
    public String toString() {
        return nome + " (" + sigla + ")";
    }
}
```

Método HTTP	URI	Descrição
POST	/cidades	Adiciona uma nova cidade

```
@Service
public class RestClientService implements IRestClientService {

    RestClient restClient = RestClient.create("http://localhost:8082"); 6 usages

    @Override 1 usage
    public Long create(Cidade cidade) {

        ResponseEntity<Cidade> res = restClient.post()
            .uri(uri: "/cidades") RequestBodySpec
            .contentType(MediaType.APPLICATION_JSON)
            .body(cidade)
            .retrieve() ResponseSpec
            .toEntity(Cidade.class);

        Cidade c = res.getBody();

        return c.getId();
    }
}
```

Método HTTP	URI	Descrição
GET	/cidades	Retorna todas as cidades

```
public List<Cidade> get() {  
    List<Cidade> list = restClient.get() RequestHeadersUriSpec<capture of ?>  
        .uri(uri: "/cidades") capture of ?  
        .accept(MediaType.APPLICATION_JSON)  
        .retrieve() ResponseSpec  
        .body(new ParameterizedTypeReference<>() {  
        });  
    return list;  
}
```

Método HTTP	URI	Descrição
GET	/cidades/{id}	Retorna a cidade de id = {id}

```
public Cidade get(Long id) {  
    Cidade cidade = restClient.get() RequestHeadersUriSpec<capture of ?>  
        .uri(uri: "/cidades/" + id) capture of ?  
        .accept(MediaType.APPLICATION_JSON)  
        .retrieve() ResponseSpec  
        .body(Cidade.class);  
    return cidade;  
}
```

Método HTTP	URI	Descrição
GET	/cidades/estados/{id}	Retorna as cidades do estado de id = {id}

```
public List<Cidade> get(Estado estado) {  
    List<Cidade> list = restClient.get() RequestHeadersUriSpec<capture of ?>  
        .uri(uri: "/cidades/estados/" + estado.getId()) capture of ?  
        .accept(MediaType.APPLICATION_JSON)  
        .retrieve() ResponseSpec  
        .body(new ParameterizedTypeReference<>() {  
        });  
    return list;  
}
```


Método HTTP	URI	Descrição
PUT	/cidades/{id}	Atualiza a cidade de id = {id}

```
public boolean update(Cidade cidade) {  
    ResponseEntity<Void> res = restClient.put() RequestBodyUriSpec  
        .uri(uri: "/cidades/" + cidade.getId()) RequestBodySpec  
        .contentType(MediaType.APPLICATION_JSON)  
        .body(cidade)  
        .retrieve() ResponseSpec  
        .toBodilessEntity();  
  
    return res.getStatusCode() == HttpStatus.OK;  
}
```


Método HTTP	URI	Descrição
DELETE	/cidades/{id}	Remove a cidade de id = {id}

```
public boolean delete(Long id) {  
    ResponseEntity<Void> res = restClient.delete() RequestHeadersUriSpec<capture of ?>  
        .uri(uri: "/cidades/" + id) capture of ?  
        .retrieve() ResponseSpec  
        .toBodilessEntity();  
    return res.getStatusCode() == HttpStatus.NO_CONTENT;  
}
```